

SECURE CAMPUS FILE SHARING & DOCUMENT MANAGEMENT SYSTEM

Project report submitted to Bharathiar University in partial fulfilment of the requirements for the award of the Degree

of

MASTER OF COMPUTER APPLICATIONS

Submitted by

MAHALINGAM S

232MCAN0099

23MCA5099

Under the guidance of

Dr. K. KARTHIKEYAN, MSc., MPhil., PhD., SET.,

Assistant Professor of Computer Science,

Department of Computer Science,

Puratchi Thalaivi Amma Government Arts and Science College, Palladam - 641664.

CENTRE FOR DISTANCE AND ONLINE EDUCATION

BHARATHIAR UNIVERSITY

COIMBATORE 641 046

MAY 2025

SECURE CAMPUS FILE SHARING & DOCUMENT MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

MAHALINGAM S

232MCAN00099

23MCA5099

In partial fulfilment of the requirements for the award of the Degree

of

MASTER OF COMPUTER APPLICATIONS

Under the guidance of

Dr. K. KARTHIKEYAN, MSc., MPhil., PhD., SET.,

Assistant Professor of Computer Science,

Department of Computer Science,

Puratchi Thalaivi Amma Government Arts and Science College, Palladam - 641664.

CENTRE FOR DISTANCE AND ONLINE EDUCATION

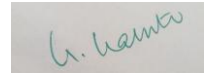
BHARATHIAR UNIVERSITY

COIMBATORE 641 046

MAY 2025

CERTIFICATE

This is to certify that the final Project work titled "**SECURE CAMPUS FILE SHARING & DOCUMENT MANAGEMENT SYSTEM**" submitted to Bharathiar University in partial fulfilment of the requirements for the award of the Degree of Master of Computer Applications is a record of the original work done by **MAHALINGAM S** under my supervision and guidance and that this project work has not formed the basis for the award of any Degree/Diploma / Associateship / Fellowship or similar title to any candidate of any University.



Programme Co-ordinator

Project Guide

Forwarded by:

Director

Centre for Distance and Online Education

Bharathiar University

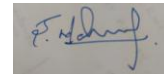
Coimbatore - 641 046

DECLARATION

I hereby declare that this project work titled "**SECURE CAMPUS FILE SHARING & DOCUMENT MANAGEMENT SYSTEM**" Submitted to the Centre for Distance and Online Education, Bharathiar University is a record of original work done by **MAHALINGAM S** under the supervision and guidance of **Dr. K. KARTHIKEYAN, MSc., MPhil., PhD., SET.,** and that this project work has not formed the basis for the award of any Degree/Diploma / Associateship / Fellowship or similar title to any candidate of any University.

Place: Coimbatore

Date:



SIGNATURE

ABSTRACT

The Secure Campus File Sharing & Document Management System is a full-stack web application developed to provide a secure and centralized platform for managing academic documents within educational institutions. In many colleges and universities, files such as assignments, lecture notes, lab records, circulars, and research documents are commonly shared through emails, messaging applications, and USB devices, which often leads to data duplication, poor organization, unauthorized access, and security risks. The proposed system aims to overcome these issues by providing a modern digital workspace for secure file storage, sharing, and management.

The application is developed using the MERN stack, which includes MongoDB, Express.js, React.js, and Node.js. The system supports Role-Based Access Control (RBAC), allowing different levels of access for students, faculty members, staff, and administrators. It includes advanced security features such as JWT authentication, Google OAuth integration, password encryption using Bcrypt, secure file upload validation, executable file blocking, NoSQL injection prevention, and obfuscated file storage using UUID-based naming techniques. The platform also maintains audit logs to track important system activities including uploads, downloads, sharing actions, and administrative operations.

The system provides several useful features such as Department Drives for departmental resource sharing, secure token-based file sharing with expiration controls, anonymous drop folders for coursework submissions, and intelligent file classification based on filenames and MIME types. The user interface is designed with a modern responsive Glassmorphism theme to ensure a premium user experience across different devices.

Overall, the project provides a scalable, secure, and efficient solution for academic file management and collaboration. It reduces dependency on insecure sharing methods, improves accessibility and organization of academic resources, and demonstrates the practical implementation of modern web technologies and secure software engineering principles in real-world educational environments.

ACKNOWLEDGEMENT

I take this opportunity to express my sincere gratitude to all those who supported me throughout the completion of this project titled "**Secure Campus File Sharing & Document Management System**".

First and foremost, I would like to thank my project guide, **Dr. K. KARTHIKEYAN, MSc., MPhil., PhD., SET.,** Assistant Professor of Computer Science, for his valuable guidance, encouragement, and continuous support throughout this project. His timely feedback and expert insights have been instrumental in shaping the project into its current form.

I would also like to thank the **Programme Co-ordinator** and the Department of Computer Science, Puratchi Thalaivi Amma Government Arts and Science College, Palladam, and the **Centre for Distance and Online Education, Bharathiar University**, for providing the necessary academic environment and infrastructure for successful project completion.

Finally, I extend my heartfelt thanks to my family and friends for their unwavering encouragement, moral support, and understanding during this endeavor.

MAHALINGAM S

Register No: 232MCAN0099

TABLE OF CONTENTS

CHAPTER / SECTION	TITLE	PAGE NO
	ABSTRACT	
	ACKNOWLEDGEMENT	
	LIST OF FIGURES / DIAGRAMS	
	LIST OF SCHEMAS / TABLES	
CHAPTER 1	INTRODUCTION	1
1.1	SYNOPSIS	1
1.2	TECHNOLOGIES USED	1
1.3	HIGH-LEVEL SYSTEM ARCHITECTURE	1
1.4	MODULE DESCRIPTION	2
CHAPTER 2	SYSTEM ANALYSIS	3
2.1	EXISTING SYSTEM	3
2.2	PROPOSED SYSTEM	3
2.3	FEASIBILITY STUDY	4
CHAPTER 3	DEVELOPMENT ENVIRONMENT	5
3.1	HARDWARE SPECIFICATIONS	5
3.2	SOFTWARE SPECIFICATIONS	5
3.3	PROGRAMMING ENVIRONMENT	5

CHAPTER 4	SYSTEM DESIGN AND IMPLEMENTATION	6
4.1	ELEMENTS OF DESIGN	6
4.1.1	PROCESS DESIGN (MULTI-LEVEL DFDS)	6
4.1.2	CONCEPTUAL DESIGN	9
4.1.3	LOGICAL DESIGN	9
4.1.4	PHYSICAL DESIGN	9
4.1.5	INPUT DESIGN	9
4.1.6	OUTPUT DESIGN	9
4.2	DATABASE DESIGN & SCHEMAS	10
4.3	HIGH-FIDELITY UI DESIGN MOCKUPS	15
CHAPTER 5	SYSTEM TESTING AND SECURITY	17
5.1	SYSTEM TESTING	17
5.2	SYSTEM SECURITY CONTROLS	19
5.3	SYSTEM ENHANCEMENT	19
CHAPTER 6	CONCLUSION AND FURTHER DIRECTIONS	20
6.1	CONCLUSION	20
6.2	FURTHER ENHANCEMENTS	20
CHAPTER 7	BIBLIOGRAPHY & REFERENCES	21

LIST OF FIGURES / DIAGRAMS

FIGURE NO	FIGURE DESCRIPTION	PAGE NO
Figure 1.1	MERN Stack High-Level System Architecture Diagram	2
Figure 4.1	Level 0 Data Flow Diagram (Context Diagram)	6
Figure 4.2	Level 1 Data Flow Diagram (Functional Process Diagram)	7
Figure 4.3	Level 2 Data Flow Diagram (Detailed Upload Flow)	8
Figure 4.4	Entity-Relationship (ER) Diagram of MongoDB Database	10
Figure 4.5	High-Fidelity UI Mockup: Student/Faculty Personal Workspace	15
Figure 4.6	High-Fidelity UI Mockup: Admin Control & Logs Console	16

LIST OF SCHEMAS / TABLES

TABLE NO	SCHEMA NAME / DESCRIPTION	PAGE NO
Table 4.1	Users Collection Schema (models/User.js)	11
Table 4.2	Files Collection Schema (models/File.js)	12
Table 4.3	Folders Collection Schema (models/Folder.js)	13
Table 4.4	Shared Links Collection Schema (models/SharedLink.js)	13
Table 4.5	Audit Logs Collection Schema (models/AuditLog.js)	14
Table 4.6	Help Requests Collection Schema (models/HelpRequest.js)	14

CHAPTER 1

INTRODUCTION

1.1 SYNOPSIS

The project titled "Secure Campus File Sharing & Document Management System" is designed specifically to facilitate secure, organized, and optimized file exchange inside educational institutions. In university setups, student and faculty directories are frequently fragmented. Traditional sharing channels, such as messaging applications and emails, introduce vulnerabilities, duplicates, and lack strict access controls. This full-stack web application is built to mitigate those challenges, providing a highly-structured digital workspace.

The system enforces Role-Based Access Control (RBAC), dividing accounts into Student, Faculty, Staff, and Administrator roles with appropriate operational scopes. Important security integrations include NoSQL query sanitization to filter out injection strings, password encryption using Bcrypt, and local storage obfuscation renaming raw files to randomized UUIDs. Furthermore, the application provides unique, powerful academic utilities such as private Department Drives, expiring shareable links, NLP-lexical document classification, and anonymous homework drop folders with strict deadlines.

1.2 TECHNOLOGIES USED

The application relies on the highly robust and scalable MERN full-stack JavaScript architecture:

- Frontend Layer: React.js (v19) combined with Vite, utilizing Tailwind CSS for custom premium Glassmorphism layouts and Lucide icons for vector representation.
- Backend Server: Node.js runtime and Express.js framework, managing Multer multi-part streams.
- Database Engine: MongoDB NoSQL Document Database mapped via Mongoose ODM.
- Security & Transmissions: JSON Web Token (JWT) session controls, Google Workspace OAuth2, and Bcrypt password hashing.

1.3 HIGH-LEVEL SYSTEM ARCHITECTURE

The system's modular layers, middleware filters, client-side boundaries, and database mappings are detailed in the High-Level System Architecture diagram below:

MERN STACK HIGH-LEVEL SYSTEM ARCHITECTURE

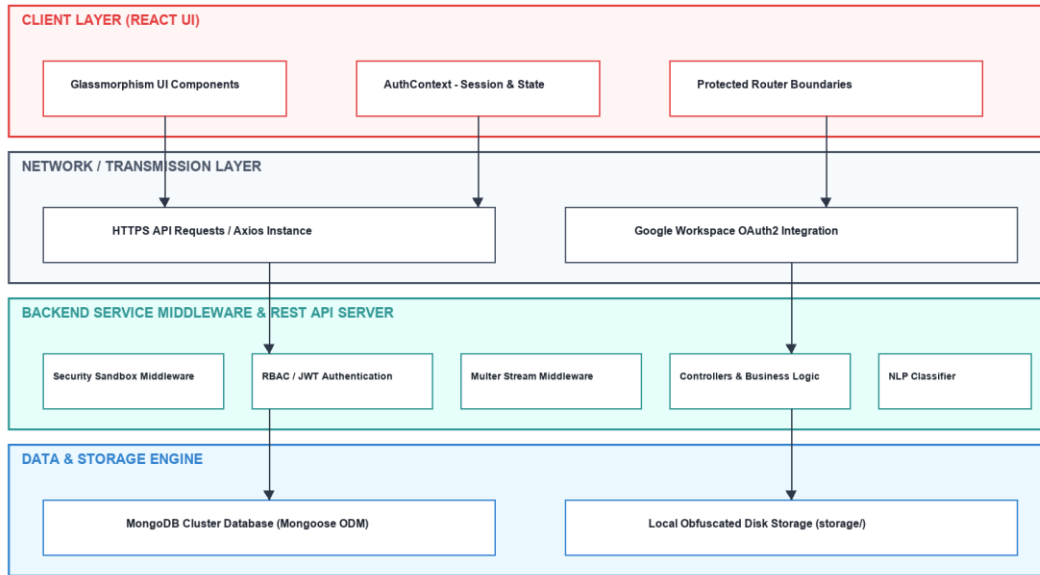


Figure 1.1: MERN Stack High-Level System Architecture Diagram

1.4 MODULE DESCRIPTION

The platform is engineered as six tightly integrated layers, guaranteeing clean code and decoupling:

1. Frontend Client Workspace Module: Built on React 19, this module coordinates user interaction across dashboards, department drives, and support interfaces. Enforces Client Router boundaries.
2. Session & Auth Module: Authenticates accounts using Bcrypt password credentials or Google Workspace SSO, issuing high-entropy JWT session tokens.
3. API Gate & Middleware Module: Acts as the primary backend guard, executing global CORS headers, NoSQL query sanitization via express-mongo-sanitize, and Multer file buffer constraints.
4. Workspace & Sharing Controller: Handles virtual directory structure creations, published Department files, secure share link tokens, and automated cron link expirations.
5. NLP Lexical Classifier Module: An asynchronous utility parsing original filenames and MIME types to auto-assign categorizations (Assignments, Lab Records, Syllabus, Lecture Notes, etc.) without manual user classification.
6. Physical Disk & Data Store: Writes raw file buffers securely inside the backend/storage/ directory under random UUID configurations while indexing all structures, profiles, and audit trails inside MongoDB.

CHAPTER 2

SYSTEM ANALYSIS

2.1 EXISTING SYSTEM

In current academic environments, document exchange is highly fragmented. Course resources, project reports, syllabi, and official notices are commonly disseminated through insecure public email lists, third-party messaging platforms, public cloud links, and physical USB drives. There is no centralized system or isolated workspace designed for educational departmental drives, and documents are scattered across separate systems.

2.1.1 DISADVANTAGES OF THE EXISTING SYSTEM

- **Lack of Centralization:** Resources are scattered, causing severe archival losses and search difficulties.
- **Absence of Role-Based Security:** Anyone with a link can modify or download files. Access is binary (public or private) with no granular student-faculty separation.
- **Severe Storage Duplication:** Multiple users upload the exact same file in separate email chains, wasting server space.
- **Malicious Executable Risks:** Public cloud drives allow users to upload executable files, exposing the institution to severe malware and injection payloads.
- **Inexistent Auditing:** Administrators have no visibility into who downloaded sensitive files, leaked resources, or deleted data.

2.2 PROPOSED SYSTEM

The proposed Secure Campus File Sharing & Document Management System provides a highly centralized, secure, and intelligent digital workspace. The platform separates users strictly into Students, Faculty, Staff, and Administrators. It provides standard departmental isolation drives and enables faculty to host anonymous coursework drop folder links with strict cut-off deadlines. Physical storage obfuscation renames files on disk, and global sanitization shields the database from common web threat vectors.

2.2.1 ADVANTAGES OF THE PROPOSED SYSTEM

- Unified Document Workspace: Combines department drives, personal workspaces, and secure sharing in a single portal.
- Granular Access Isolation: Restricts operations to authorized student-faculty role parameters.
- Storage Obfuscation & Traversal Shielding: Strips file extensions and renames files on disk, nullifying target URL hacking.
- Intelligent Auto-Categorization: Leverages an integrated classifier to structure documents without manual entry.
- Immutable Security Logging: Creates tamper-proof activity logs for all upload, download, and delete actions.

2.3 FEASIBILITY STUDY

A feasibility study evaluates the system requirements, development factors, and institutional impacts to guarantee successful platform execution:

- Technical Feasibility: The application uses the highly standard MERN stack. Since Node.js, React, and MongoDB Atlas are globally mature and highly documented, the technical environment is highly stable and feasible.
- Operational Feasibility: The Glassmorphism UI is designed to be highly responsive and user-friendly. Academic staff and students can easily adapt to directory navigations, breadcrumb trails, and drop link creations without training.
- Economic Feasibility: The platform uses open-source software and databases, requiring zero licensing costs. It can run on basic local university server grids (Intel i3, 4GB RAM) or scale easily to the cloud, making it economically practical.

CHAPTER 3

DEVELOPMENT ENVIRONMENT

3.1 HARDWARE SPECIFICATIONS

The development and execution environment demands very low physical hardware parameters:

- Processor: Intel Core i3 or higher (or AMD equivalent)
- Random Access Memory (RAM): 4 GB or higher
- Storage Disk Capacity: 500 GB or higher HDD/SSD
- Display Resolution: Standard 1024x768 monitor screen or above
- Connectivity: Internet connection (needed for Google OAuth and MongoDB Atlas cloud maps)

3.2 SOFTWARE SPECIFICATIONS

The project is built on mature open-source software packages:

- Operating System: Windows 10/11, macOS, or Linux server nodes
- Client Platform: React.js (v19) running on Vite bundlers
- Server Framework: Node.js runtime and Express.js server frameworks
- Data Layer: MongoDB database mapped via Mongoose ODM library
- Security Tools: JSON Web Token keys, Google OAuth2 client, express-mongo-sanitize, and Bcrypt.
- Package Manager: npm (Node Package Manager) v10.0.0 or higher

3.3 PROGRAMMING ENVIRONMENT

The programming environment leverages full-stack JavaScript integration, eliminating cross-compilation layers and optimizing data-mapping speed: Node.js manages execution loops, Express maps routes, MongoDB provides document flexibility, and React structures highly dynamic and responsive frontends. Security libraries handle token validations, query sanitizations, and file-parsing filters at the gate, forming a secure enterprise-grade architecture.

CHAPTER 4

SYSTEM DESIGN AND SPECIFICATION

4.1 ELEMENTS OF DESIGN

System design defines the structural architecture, data pathways, model mappings, and user-facing layouts to secure platform development.

4.1.1 PROCESS DESIGN (MULTI-LEVEL DFDS)

Data Flow Diagrams (DFDs) detail the movement of information through the platform, mapping how raw input data streams from external entities, processes inside modular boundaries, registers in stores, and generates outputs.

A. LEVEL 0 DFD (CONTEXT DIAGRAM)

The Context Diagram maps high-level boundaries between actors and the centralized platform core:

LEVEL 0 DFD (CONTEXT DIAGRAM)

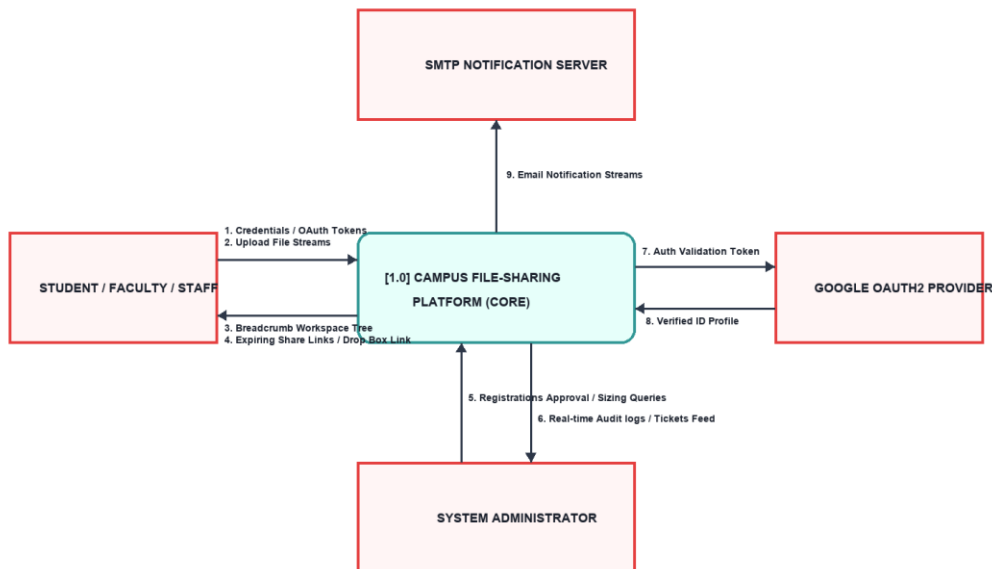


Figure 4.1: Level 0 DFD (Context Diagram)

B. LEVEL 1 DFD (FUNCTIONAL PROCESS DIAGRAM)

The Level 1 DFD details information movement and storage writes across the five core system bubbles:

LEVEL 1 DFD (FUNCTIONAL PROCESS DIAGRAM)

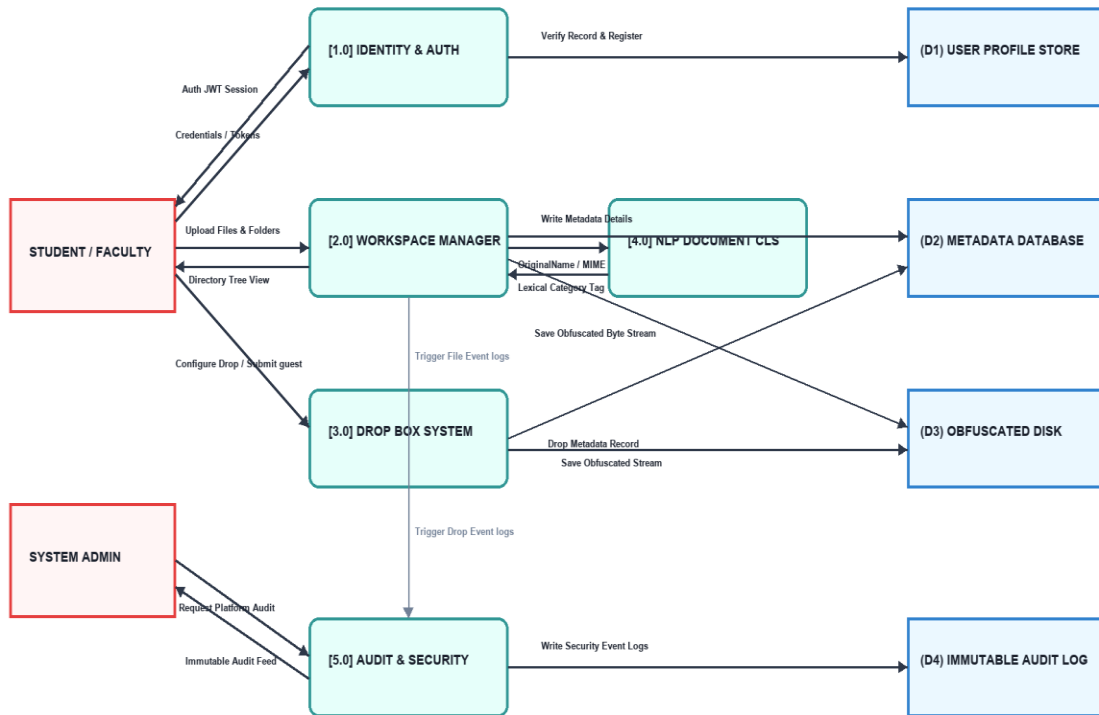


Figure 4.2: Level 1 DFD (Functional Process Diagram)

C. LEVEL 2 DFD (DETAILED FILE UPLOAD & VERIFICATION PROCESS)

The Level 2 DFD tracks raw upload bytes passing size, extension, and classification gates:

LEVEL 2 DFD (FILE UPLOAD & SECURITY VERIFICATION)

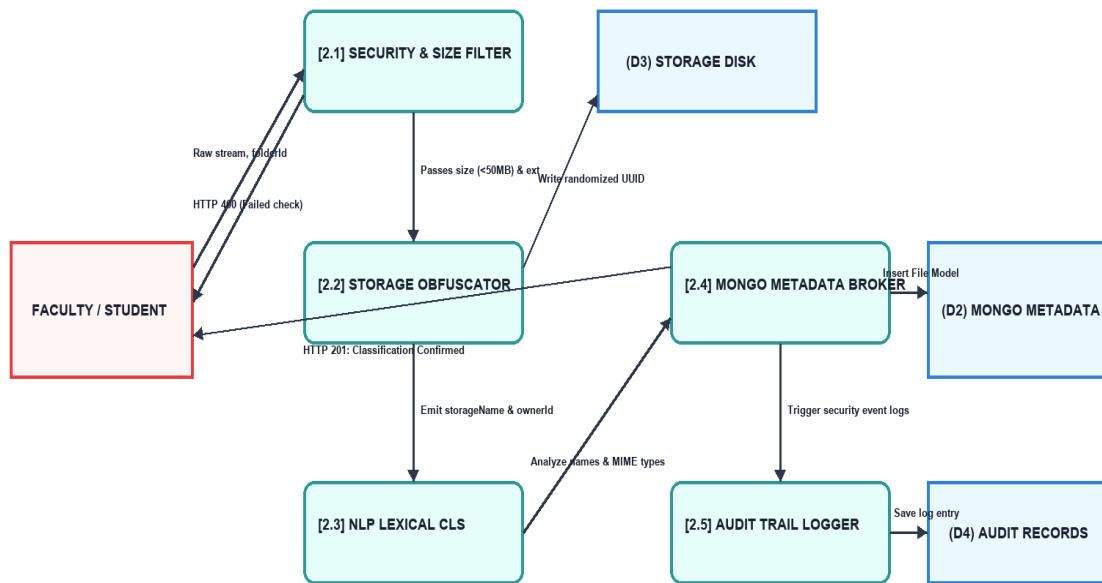


Figure 4.3: Level 2 DFD (Detailed File Upload & Verification Process)

4.1.2 CONCEPTUAL DESIGN

The conceptual design models role interactions: Students manage private files, publish to department grids, and generate shareable links. Faculty publish course resources and host deadline-controlled homework drop folders. Staff manage general circulars, while Admins approve registrations, stamp verified documents, and inspect logs.

4.1.3 LOGICAL DESIGN

Logical design models abstract components: React 19 Client UI with JWT AuthContext boundaries; Express REST API gateways processing query sanitizations and Multer multi-part streams; and the NLP Lexical Classifier mapping filenames and MIME type descriptors into categorized tag fields.

4.1.4 PHYSICAL DESIGN

Physical design implements structural elements: React 19 and Tailwind CSS styling high-contrast Glassmorphism panels; Node/Express backend servers running Bcrypt password security; MongoDB storing records via Mongoose ODM schemas; and local obfuscated disks in backend/storage/ writing streams under random UUIDs with stripped file extensions.

4.1.5 INPUT DESIGN

Input design validates forms: User Registration checks email regex, role selectors, and rolls; Secure File Upload filters sizes (<50MB) and blocks executable extensions; Drop Box creation validates cut-off dates and times.

4.1.6 OUTPUT DESIGN

Output design renders feeds: breadcrumb-navigated file listings, color-coded classification badges, tabular admin log consoles, and platform usage analytics.

4.2 DATABASE DESIGN & SCHEMAS

The data storage layer maps collections using MongoDB document schemas managed via Mongoose ODM. Relational linkages, indices, PK/FK configurations, and cardinality parameters are modeled below:

ENTITY-RELATIONSHIP (ER) DIAGRAM - MONGO DATABASE

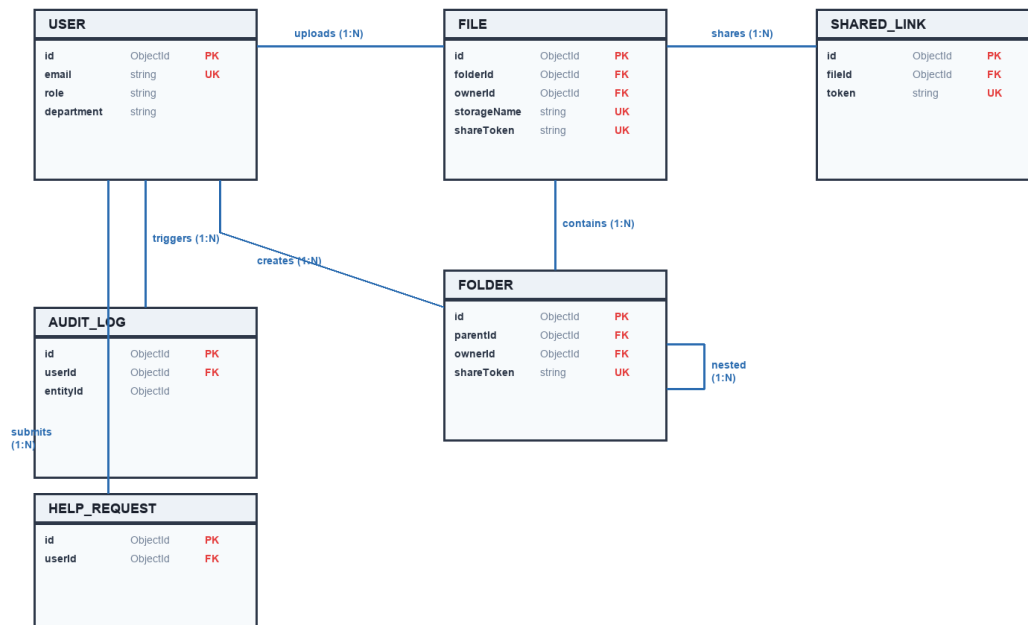


Figure 4.4: MongoDB Database Entity-Relationship Diagram

TABLE 4.1: USERS COLLECTION SCHEMA (MODELS/USER.JS)

Field Name	Data Type	Constraints / Defaults	Description
name	String	Required	Full name of the user
email	String	Required, Unique, Regexp Validated	Institutional email address
password	String	Required, Min length: 6, select: false	Hashed credential using Bcrypt (10 salt rounds)
role	String	Enum: ['student', 'faculty', 'staff', 'admin'], Default: 'student'	Role-Based Access Control level
department	String	Enum: ['CSE', 'MCA', 'ECE', 'Placement Cell', 'Examination Cell', 'General'], Default: 'General'	Primary academic department division
rollNo	String	Default: null	Student roll number or staff employee ID
isApproved	Boolean	Default: true	Approval gate controlled by Administrators
timestamps	Boolean	Automatic	Logs createdAt and updatedAt dates

TABLE 4.2: FILES COLLECTION SCHEMA (MODELS/FILE.JS)

Field Name	Data Type	Constraints / Defaults	Description
originalName	String	Required	User's file name at upload time
storageName	String	Required, Unique	Renamed physical file identifier (UUID)
folderId	ObjectId	Ref: Folder, Default: null	Parent folder identification
ownerId	ObjectId	Ref: User, Required	Owner who uploaded the resource
mimeType	String	Required	Detected MIMEtype (e.g., application/pdf)
category	String	Default: 'Uncategorized'	Auto-classification category output
department	String	Default: 'General'	Department assigned for access isolation
isPublicToDepartment	Boolean	Default: false	Visible on the Department Drive if true
sizeBytes	Number	Required	Physical size of the file in bytes
shareToken	String	Sparse, Unique	Secure randomized sharing token
isVerified	Boolean	Default: false	Signed verification mark by Administrator
shareExpiresAt	Date	Default: null	Expiry timestamp for sharing link
shareDownloadLimit	Number	Default: null	Download count threshold cap
shareDownloadCount	Number	Default: 0	Live download tracking

TABLE 4.3: FOLDERS COLLECTION SCHEMA (MODELS/FOLDER.JS)

Field Name	Data Type	Constraints / Defaults	Description
name	String	Required	Human-readable directory name
parentId	ObjectId	Ref: Folder, Default: null	Self-referential ID for tree structures
ownerId	ObjectId	Ref: User, Required	Creator of the folder
shareToken	String	Sparse, Unique	Sharing token for anonymous submittals
isDropFolder	Boolean	Default: false	Flags folder as an active Drop Box
deadline	Date	Default: null	Cut-off timestamp for Drop submissions
department	String	Default: 'General'	Primary department isolation
isPublicToDepartment	Boolean	Default: false	True makes it public on Department Drive

TABLE 4.4: SHARED LINKS COLLECTION SCHEMA (MODELS/SHAREDLINK.JS)

Field Name	Data Type	Constraints / Defaults	Description
fileId	ObjectId	Ref: File, Required	Target file mapping
token	String	Required, Unique	Generated link path parameter
expiresAt	Date	Default: null	Auto-expiration time
accessLimitRole	String	Enum: ['student', 'faculty', 'admin', 'all'], Default: 'all'	Specific roles authorized to open the link

TABLE 4.5: AUDIT LOGS COLLECTION SCHEMA (MODELS/AUDITLOG.JS)

Field Name	Data Type	Constraints / Defaults	Description
userId	ObjectId	Ref: User, Required	User triggering the system action
action	String	Enum: ['upload', 'download', 'delete', 'share', 'create_folder', 'delete_account']	Type of interaction performed
entityType	String	Enum: ['file', 'folder', 'user']	Category of target object
entityId	ObjectId	Required	Database primary key of target object
details	String	Optional description	Extended event context details
ipAddress	String	Captured on HTTP Request	IP Address tracking of initiator

TABLE 4.6: HELP REQUESTS COLLECTION SCHEMA (MODELS/HELPREQUEST.JS)

Field Name	Data Type	Constraints / Defaults	Description
userId	ObjectId	Ref: User, Required	User who filed the ticket
subject	String	Required	Brief description of issue
message	String	Required	Comprehensive problem details
type	String	Enum: ['General', 'Technical Issue', 'Student Account Issue', 'Feature Request']	Support category
status	String	Enum: ['Pending', 'In Progress', 'Resolved'], Default: 'Pending'	Resolution status

4.3 HIGH-FIDELITY UI DESIGN MOCKUPS

The interface utilizes custom Glassmorphism components layered over animatable mesh backgrounds, with colorful auto-classification badges and structured grid panels:

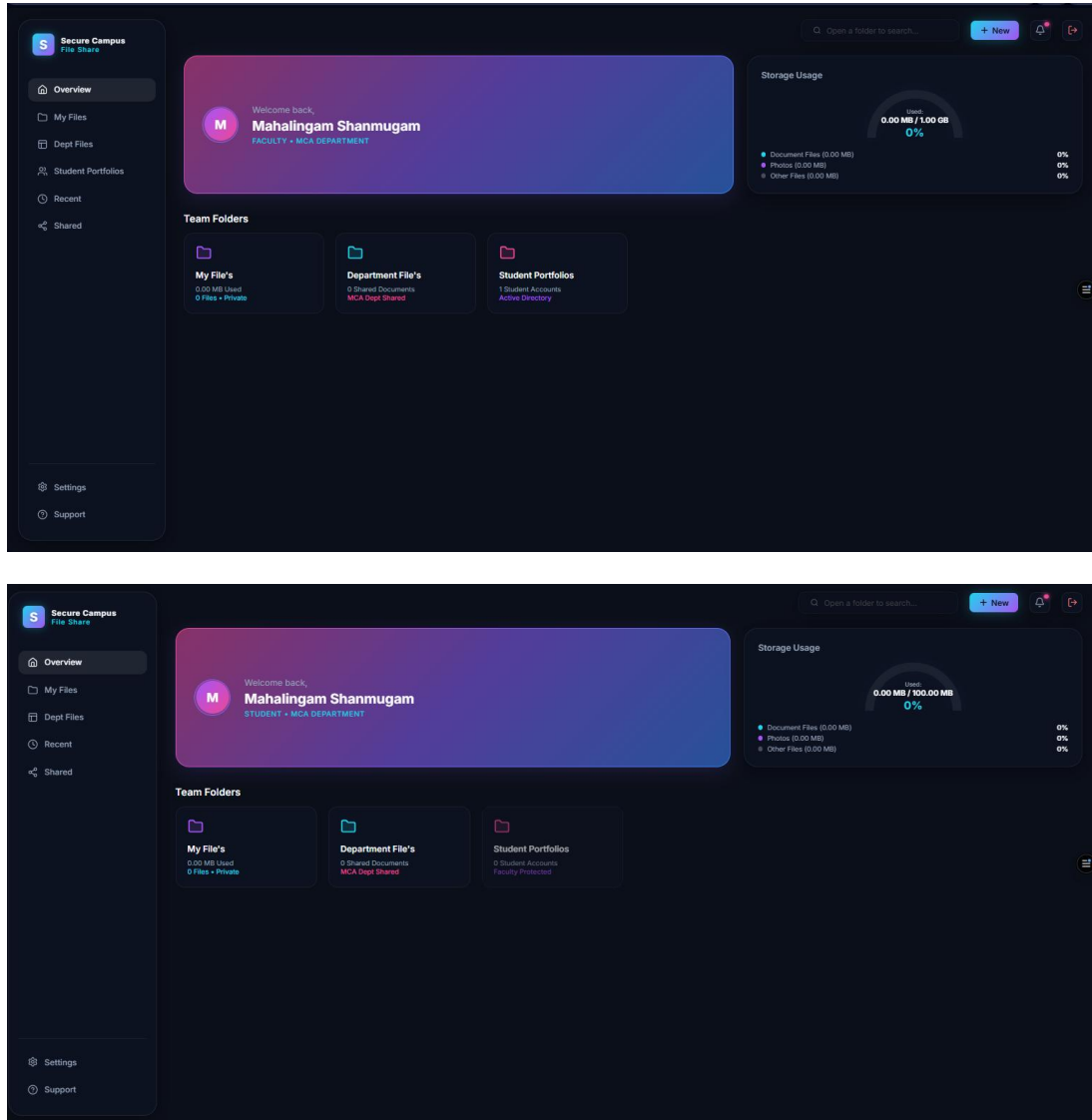


Figure 4.5: High-Fidelity UI Mockup: Student / Faculty Workspace Dashboard

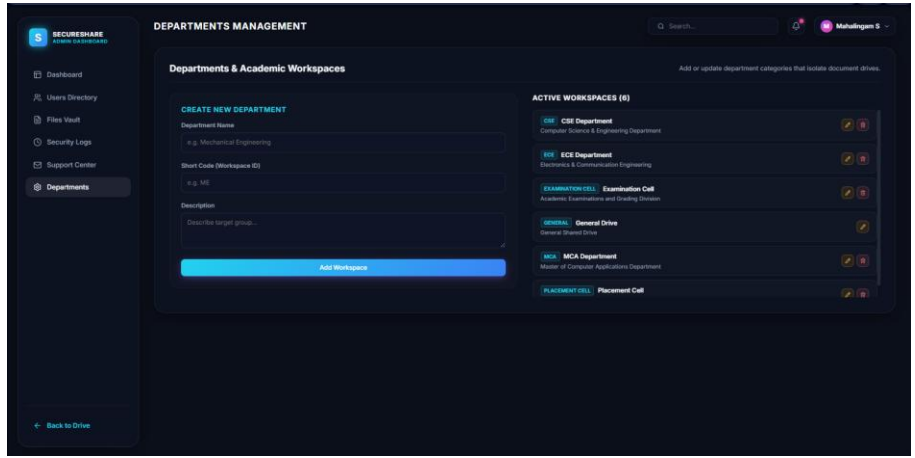
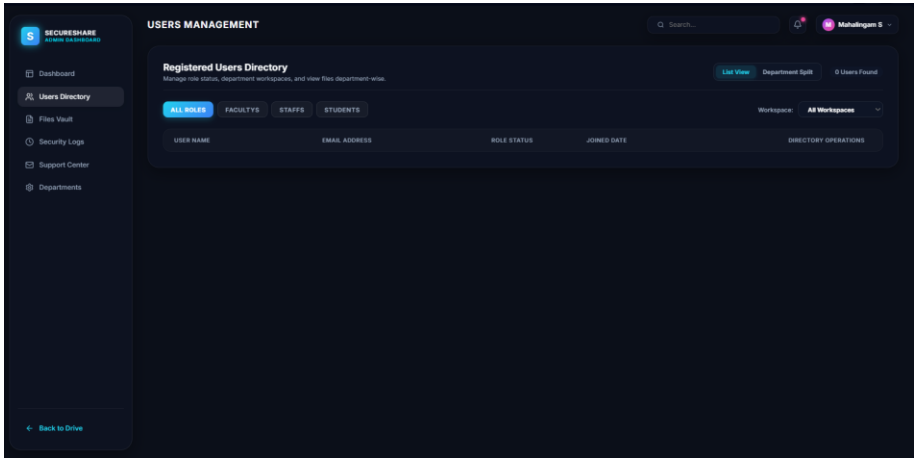
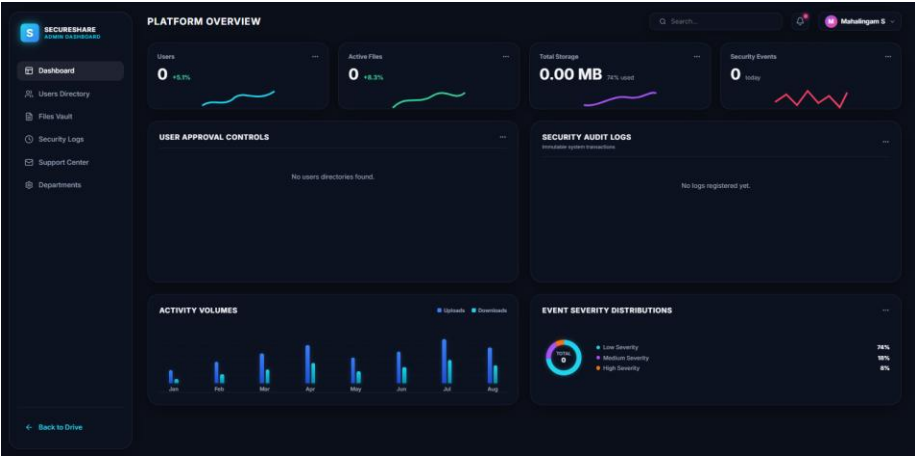


Figure 4.6: High-Fidelity UI Mockup: Admin Control Panel & Log Traverser

CHAPTER 5

SYSTEM TESTING AND SECURITY

5.1 SYSTEM TESTING

System testing is a crucial phase in software engineering that evaluates the fully integrated application against user requirements and strict security specs. In the Secure Campus File Sharing & Document Management System, testing was conducted systematically across multiple testing layers to guarantee performance, modular security, and absolute system reliability.

5.1.1 UNIT TESTING

Unit testing isolates and tests the smallest components of the codebase independently:

- Lexical Document Classifier (utils/classifier.js): Verified that strings such as 'MCA-Lab1-Experiment.pdf' correctly resolve to 'Lab Records' and that mime-types like 'application/vnd.ms-excel' map to 'Spreadsheets'.
- Security Extension Filter: Ensured that the backend's Multer file filter helper immediately blocks and rejects extensions like '.exe', '.sh', '.bat', and '.cmd', returning appropriate HTTP 403 status codes.
- Bcrypt Hashing Pre-Save Hook: Confirmed that User models encrypt passwords on save and that Mongoose correctly hides hashed values on standard find queries (select: false).

5.1.2 INTEGRATION TESTING

Integration testing evaluates combined modules and data flows:

- Anonymous Drop Box Upload Pipeline: Checked the sequence where a guest uploads a document to `/api/folders/drop/:token`. Verified that the file is obfuscated, written to storage/, and that a corresponding MongoDB File document is safely inserted under the faculty owner's workspace with the target folderId.
- JWT Auth Lifecycle & Header Transmission: Verified that login payloads successfully return JWT tokens and that client-side Axios instances dynamically inject 'Bearer <token>' inside headers on subsequent protected requests.
- Virtual Folder Cascade Delete: Verified that deleting a parent folder triggers the backend controllers to delete all children files physically on disk and wipe out metadata records in MongoDB.

5.1.3 USER ACCEPTANCE TESTING (UAT)

User Acceptance Testing validates workflows against real student-faculty academic scenarios:

- **Student Submissions:** Tested the user experience for students uploading homework assignments, verifying that the Glassmorphism workspace rendered responsive breadcrumbs and file categorizations in real time.
- **Faculty Document Dissemination:** Verified that teachers could easily upload course syllabi and circulars, publish them to the CSE Drive, configure Drop boxes, and easily download coursework packages.
- **Administrative Controls:** Confirmed that administrators could seamlessly approve registered student accounts, manage support requests, and inspect audit activity feeds.

5.1.4 OUTPUT TESTING

Output testing ensures that correct, decrypted stream download responses are dispatched:

- **File Decryption & Download Stream:** Confirmed that clicking download requests retrieves the physical UUID-based storage file, restores its original name and MIME type from the database, and streams the buffer seamlessly to the user.
- **Notification Dispatch:** Verified that publishing an official announcement successfully triggers SMTP notification emails to all students associated with that specific department division.

5.1.5 VALIDATION TESTING

Validation testing asserts limits, dates, and query parameters at the gates:

- **File Sizing Enforcer:** Checked that files larger than 50MB are rejected by Express with an HTTP 400 Payload Too Large.
- **NoSQL Sanitization check:** Verified that body parameter strings representing query selectors like `'{"$gt": ""}'` are stripped off by `express-mongo-sanitize` to completely prevent NoSQL injection attacks.
- **Expiration Validation:** Confirmed that downloading files using shared link tokens beyond their `expiresAt` datetime returns HTTP 410 Expired Link.

5.2 SYSTEM SECURITY CONTROLS

The system incorporates high-strength defensive mechanisms to nullify attack vectors:

- NoSQL Injection Blocking: Globally scrubs body and query variables to strip off injection vectors (like \$ and .).
- Local Storage Obfuscation: Strips physical extensions and writes uploaded files under randomized UUID configurations, blocking direct target URL hacking.
- Cryptographic Safety: Password parameters are encrypted with Bcrypt (10 salt rounds), and API routes are secured under high-entropy JWT secrets.
- Granular RBAC validation: Backend authorization middleware checks that students, faculty, staff, and admin accounts execute matching API scopes.
- Short-lived Expirations: Tokenized sharing paths utilize sparseness, download limits, and strict expiresAt cut-offs.

5.3 SYSTEM ENHANCEMENT

Future development roadmaps include high-level optimizations:

- Hybrid Cloud Bucket Integration: Direct pre-signed multi-part uploads to AWS S3 or Google Cloud storage nodes.
- Real-time ClamAV Scan: Background scanning of upload streams before committing disk writing.
- PDF.js Previews: Safe, in-browser rendering of documents inside dashboard panels.

CHAPTER 6

CONCLUSION AND FURTHER DIRECTIONS

6.1 CONCLUSION

The Secure Campus File Sharing & Document Management System was successfully designed, developed, and tested on the modern MERN stack. It resolves the challenges of fragmented and insecure email-based academic document distribution. By consolidating uploads into isolated workspaces, nested folder trees, and public department drives, the application significantly improves collaborative workflow, decreases administrative overhead, and guarantees document preservation.

Furthermore, the integration of automated lexical classifications and high-strength security architectures (Bcrypt, JWT boundaries, NoSQL sanitization filters, and UUID disk storage obfuscation) secures the confidentiality, integrity, and accessibility of critical academic resources. The system represents a scalable, robust, and enterprise-grade digital workspace suitable for real-world universities.

6.2 FURTHER ENHANCEMENTS

Future developmental versions can expand platform features:

- Institutional Single Sign-On (SSO): Complete integration with LDAP or Active Directory systems.
- OCR Semantic Indexing: Running background text extraction on images and PDFs to support full-text search capabilities.
- Multi-user Collaborative Editing: Integrating real-time collaborative document editors.

CHAPTER 7

BIBLIOGRAPHY & REFERENCES

1. Abraham Silberschatz, Henry Korth, S. Sudarshan – “Database System Concepts”, 7th Edition, McGraw-Hill.
2. Roger S. Pressman, Bruce R. Maxim – “Software Engineering: A Practitioner’s Approach”, 9th Edition, McGraw-Hill.
3. David Flanagan – “JavaScript: The Definitive Guide”, 7th Edition, O'Reilly Media.
4. Ramez Elmasri, Shamkant B. Navathe – “Fundamentals of Database Systems”, 7th Edition, Pearson.
5. MongoDB Official Documentation, Online: <https://www.mongodb.com/docs>
6. React Framework Documentation, Online: <https://react.dev>
7. Express Server Guide, Online: <https://expressjs.com>
8. Node.js Runtime Reference, Online: <https://nodejs.org>
9. OWASP Top Ten Web Application Security Risks, Online: <https://owasp.org>